DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

Stack-Augmented Decoding for Real-World Prompt Decomposition and Visualization

Data Structures and Applications – IS233AI

Experiential Learning (Lab)

REPORT

Submitted by

A V Kruthi Krishna 1RV24IS001

Aayushi Priya 1RV24IS005

Under the guidance of

Prof. Kavitha S N

Assistant Professor

Department of Information Science Engineering

Affiliation

In partial fulfilment for the award of degree of

Bachelor of Engineering

in

Department of Information Science and Engineering

2025-2026

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

CERTIFICATE

Certified that the project work titled **'Stack-Augmented Decoding for Real-World Prompt Decomposition and Visualization'** is carried out by **A V Kruthi Krishna(1RV24IS001) and Aayushi Priya (1RV24IS005)** who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfilment for the award of degree of **Bachelor of Engineering in Information Science and Engineering** of the **Visvesvaraya Technological University**, Belagavi during the academic year 2025-2026. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report deposited in the departmental library. The report has been approved as it satisfies the academic requirements in respect of experiential learning work prescribed by the institution for the said degree.

Signature of Guide — **Signature of Head of the Department**

Prof. Kavitha S N — **Dr. G S Mamatha**

External Viva

Name of Examiners — **Signature with Date**

1
2

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

DECLARATION

We**, A V Kruthi Krishna and Aayushi Priya ,** students of Third semester B.E., department of Information Science Engineering, RV College of Engineering, Bengaluru, hereby declare that Experiential Learning (Lab) titled **'Stack-Augmented Decoding for Real-World Prompt Decomposition and Visualization'** has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in **Information Science and Engineering** during the academic year 2025-26

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering®, Bengaluru and we will be one of the authors of the same.

Place: Bengaluru
Date:

Name	**Signature**
1. A V KRUTHI KRISHNA(1RV24IS001)	
2. AAYUSHI PRIYA (1RV24IS005)	

ABSTRACT

Modern intelligent systems are capable of processing complex user prompts; however, the internal reasoning process used to generate outputs is often hidden from learners. This makes it difficult for students to relate theoretical concepts of Data Structures, particularly stacks, to their practical applications. Existing learning tools largely focus on final outputs and use built-in abstractions, which limits visibility into execution order and reasoning flow. As a result, learners struggle to understand how structured problem decomposition and execution actually occur in real-world systems.

This project addresses the above limitation by proposing a Stack-Augmented Prompt Decomposition and Reasoning Visualization system that explicitly applies the stack data structure to manage and visualize reasoning steps. The system takes a user-provided prompt and decomposes it into smaller logical tasks. These tasks are pushed onto a stack and executed using the Last-In-First-Out (LIFO) principle. Each push and pop operation corresponds to a reasoning step, allowing learners to clearly observe execution order, task dependencies, and stack behavior.

The system is implemented using a stack-based control mechanism and a visual interface that displays stack contents, stack height, execution progress, and step-by-step reasoning. Unlike conventional systems that treat reasoning as a black box, this approach provides a transparent view of how tasks are structured and processed. The visualization of stack operations such as push, pop, and empty-stack completion plays a crucial role in reinforcing theoretical DSA concepts.

The results demonstrate that the proposed system effectively bridges the gap between stack theory and practical application. By making reasoning execution explicit and visual, the system enhances conceptual understanding and learner engagement. Compared to traditional static learning methods, the proposed approach is more interactive, intuitive, and educational. In conclusion, this project successfully connects data structure theory with practical reasoning workflows, and future enhancements may include support for additional data structures and more advanced visualization techniques.

TABLE OF CONTENTS

	PAGE NO
Abstract	**I**
Chapter 1: Introduction	**6**
Chapter 2: Solution Design	**8**
Chapter 3: Implementation Details.	**10**
Chapter 4: Results and Discussion	**12**
Chapter 5: Conclusion and Future Scope	**15**
References	16
Appendix	**17**

CHAPTER 1 - Introduction

Preamble

This chapter introduces the background, motivation, and purpose of the project titled "Stack-Augmented Prompt Decomposition and Reasoning Visualization." It provides an overview of the problem being addressed, explains the need for structured reasoning in modern systems, and highlights the relevance of classical data structures in solving real-world problems. The chapter sets the foundation for understanding how stack-based execution can improve clarity, transparency, and learning outcomes.

1.1 Introduction

Modern computational systems are capable of handling complex user inputs, yet the internal process used to arrive at a solution often remains hidden. In educational environments, especially in Data Structures and Algorithms (DSA), understanding the process is as important as obtaining the final result. When reasoning steps are not visible, it becomes difficult to analyze, debug, or learn from the system's behavior.

The project Stack-Augmented Prompt Decomposition and Reasoning Visualization addresses this issue by applying the stack data structure to control and visualize reasoning flow. The system breaks down a complex prompt into smaller tasks and executes them using the Last-In-First-Out (LIFO) principle, making the reasoning process explicit and easy to follow.

1.2 Background

Stacks are one of the most fundamental data structures in computer science. They are widely used in function calls, expression evaluation, undo operations, and recursion. Despite their importance, many applications of stacks remain abstract to students.

By using a stack to manage reasoning tasks, this project demonstrates a practical and intuitive application of stack operations such as push, pop, and stack height tracking. The reasoning steps are displayed visually, helping learners relate theoretical DSA concepts to real-world problem solving.

1.3 Problem Statement

Most existing systems generate answers without exposing intermediate reasoning steps. This results in:

- Lack of transparency
- Difficulty in understanding execution order
- Limited educational value

There is a need for a system that can decompose complex prompts, execute tasks in a structured manner, and visually represent the reasoning process using classical data structures.

1.4 Objective of Project

The main objectives of this project are:

- To decompose complex prompts into smaller logical tasks
- To apply stack-based (LIFO) execution for reasoning
- To visualize stack operations and reasoning steps
- To improve explainability and learning outcomes
- To demonstrate real-world applications of DSA concepts

1.5 Scope of Project

The scope of this project is limited to demonstrating stack-based task decomposition and reasoning visualization. The system focuses on real-world planning and explanation tasks and does not involve external data retrieval or advanced mathematical computation. The project is designed primarily for educational and academic evaluation purposes.

CHAPTER 2 – Solution Design

Preamble

This chapter presents the design approach for the Stack-Augmented Prompt Decomposition and Reasoning Visualization system. It outlines the system architecture, the role of the stack data structure in managing reasoning tasks, and the overall workflow from prompt input to final output generation.

2.1 System Architecture

The system follows a modular architecture consisting of three main components:

Input Processing Module: Accepts user prompts and prepares them for decomposition.

Stack-Based Execution Engine: Manages task decomposition, stack operations, and reasoning execution using the LIFO principle.

Visualization Module: Provides real-time visual representation of stack contents, operations, and reasoning flow.

2.2 Data Structure Selection

The stack data structure was selected as the core component for the following reasons:

- LIFO property naturally models depth-first task execution
- Simple operations (push, pop, peek) make implementation straightforward
- Stack height tracking provides clear progress indication
- Well-established theoretical foundation aids learning objectives

2.3 Workflow Design

After The system workflow consists of the following stages:

Stage 1 - Prompt Analysis: The user prompt is analyzed to identify its complexity and structure.

Stage 2 - Task Decomposition: The prompt is broken down into smaller logical subtasks representing distinct reasoning steps.

Stage 3 - Stack Initialization: All decomposed tasks are pushed onto the stack in reverse order of execution.

Stage 4 - Sequential Execution: Tasks are popped from the stack one by one and executed, with each step contributing to the final reasoning.

Stage 5 - Visualization Update: After each operation, the visualization is updated to reflect current stack state and execution progress.

Stage 6 - Output Generation: When the stack becomes empty, all reasoning steps are consolidated into a final output.

2.4 Design Considerations

Transparency: Every stack operation is made visible to enhance understanding of the execution process.

Educational Value: The system prioritizes clarity and learning over optimization, making each step explicit.

Scalability: The design allows for extension to other data structures such as queues and trees in future versions.

CHAPTER 3 - Implementation Details

Preamble

This chapter describes the implementation details of the project "Stack-Augmented Prompt Decomposition and Reasoning Visualization." It explains how the system is designed and executed using the stack data structure to manage reasoning tasks. The chapter focuses on how prompt decomposition, stack operations, and visualization are implemented to achieve structured and transparent reasoning.

3.1 Overview of Implementation

The implementation of this project is centered on applying the stack data structure to control the execution of reasoning tasks. The system takes a user prompt, breaks it into smaller logical tasks, and executes these tasks using the Last-In-First-Out (LIFO) principle. Each step of execution is displayed visually, allowing users to understand how the final output is generated.

3.2 Prompt Input and Task Decomposition

The implementation begins with accepting a user prompt through the interface. The prompt may describe a real-world activity, planning task, or conceptual explanation. Once received, the prompt is analyzed and divided into smaller tasks such as identifying goals, understanding constraints, organizing steps, and forming a final explanation. Each of these tasks represents one reasoning step and is prepared for stack-based execution.

3.3 Stack-Based Task Execution

After task decomposition, all tasks are pushed onto a stack. The system strictly follows the Last-In-First-Out (LIFO) principle, where the most recently added task is executed first. During execution:

- Tasks are popped one by one from the stack
- Each pop operation triggers a reasoning step
- The current stack height is tracked
- Execution progress is updated continuously

This approach ensures a clear and structured execution flow, highlighting the practical use of stack operations such as push and pop.

3.4 Reasoning Visualization

To improve clarity and understanding, the system visually represents the reasoning process. The visualization includes:

- Stack contents during execution
- Stack height changes over time
- Order of task execution
- Step-by-step reasoning output

This visual representation helps users easily connect theoretical stack concepts with real-world reasoning behavior.

3.5 Final Output Generation

Once all tasks have been executed and the stack becomes empty, the system combines the results of individual steps into a final consolidated output. This output is presented in a simple and structured manner, clearly reflecting the reasoning path followed by the system.

3.6 Technical Implementation

The system uses standard stack operations implemented through array-based or linked-list-based structures. Key functions include:

- push(task): Adds a new task to the top of the stack
- pop(): Removes and returns the task at the top of the stack
- peek(): Views the top task without removing it
- isEmpty(): Checks if the stack is empty
- size(): Returns the current stack height

CHAPTER 4: RESULTS AND DISCUSSION

Preamble

This chapter presents the results obtained from the Stack-Augmented Prompt Decomposition and Reasoning Visualization system. It discusses the effectiveness of the stack-based approach in achieving transparent reasoning execution and analyzes how the visualization enhances understanding of DSA concepts.

4.1 System Functionality

The implemented system successfully demonstrates the following capabilities:

Task Decomposition: Complex prompts are accurately broken down into logical subtasks that represent distinct reasoning steps.

Stack Operations: All stack operations (push, pop, peek) are executed correctly following the LIFO principle.

Execution Flow: Tasks are processed in the correct order, with each step building upon previous results.

Visualization: Real-time display of stack contents and operations provides clear insight into the reasoning process.

4.2 Educational Value Assessment

The system effectively serves its educational purpose by:

Making Abstract Concepts Concrete: Students can observe how theoretical stack operations apply to real-world problem-solving tasks.

Enhancing Understanding: The step-by-step visualization helps learners understand execution order and task dependencies.

Demonstrating LIFO Principle: The system clearly shows how the Last-In-First-Out property governs task execution.

Bridging Theory and Practice: The connection between DSA concepts and practical applications becomes evident through the visualization.

4.3 Comparison with Traditional Approaches

Compared to conventional learning methods, the proposed system offers several advantages:

Transparency: Unlike black-box systems, every reasoning step is visible and traceable.

Interactivity: Users can observe real-time changes in stack state during execution.

Engagement: Visual representation makes learning more engaging than text-based explanations.

Intuition Building: Seeing stack operations in action helps build stronger conceptual understanding.

4.4 Observations

Key observations from the implementation include:

Effectiveness of Visualization: The visual display of stack operations significantly improves comprehension of the LIFO principle.

Clarity of Execution: Breaking down complex prompts into smaller tasks makes the reasoning process easier to follow.

Learning Impact: Students can better understand how stacks manage execution flow in real applications.

Scalability Potential: The design can be extended to demonstrate other data structures with minimal modifications.

4.5 Limitations

The current implementation has the following limitations:

Scope: The system focuses only on stack-based reasoning and does not include other data structures.

Complexity: The system handles educational examples and may not scale to highly complex real-world scenarios.

Optimization: The implementation prioritizes clarity over performance optimization.

CHAPTER 5: CONCLUSION AND FUTURE SCOPE

Preamble

This chapter presents the conclusion of the project "Stack-Augmented Prompt Decomposition and Reasoning Visualization." It summarizes the work carried out, highlights the key outcomes of the implementation, and reflects on how the project meets its objectives using fundamental Data Structures concepts. The chapter also discusses potential future enhancements.

5.1 Conclusion

The project Stack-Augmented Prompt Decomposition and Reasoning Visualization successfully demonstrates how a classical data structure, namely the stack, can be applied to decompose complex prompts and execute reasoning in a structured manner. By following the Last-In-First-Out (LIFO) principle, the system ensures clear execution order and transparent reasoning flow.

The system effectively breaks down a user prompt into smaller logical tasks, processes them step by step using stack operations, and visualizes the entire reasoning process. This approach improves explainability and helps users understand how each step contributes to the final output.

From a Data Structures and Algorithms perspective, the project provides a practical and intuitive application of stack operations such as push, pop, and stack height tracking. The visualization component bridges the gap between theoretical concepts and real-world applications, making the learning process more engaging and meaningful.

Overall, the project fulfills its objectives by combining structured task execution with clear visual representation, thereby enhancing both understanding and educational value.

5.2 Key Achievements

The project successfully accomplished the following:

- Demonstrated practical application of stack data structure in reasoning systems
- Created a transparent visualization of stack operations and execution flow
- Developed an educational tool that enhances DSA concept understanding
- Implemented LIFO-based task decomposition and execution mechanism
- Established a foundation for future extensions to other data structures

5.3 Future Enhancements

The project can be further enhanced in the following ways:

Support for Additional Data Structures: Extend the system to demonstrate queues, trees, graphs, and other DSA concepts with similar visualization techniques.

Complex Prompt Handling: Implement more sophisticated decomposition algorithms to handle highly complex and nested reasoning tasks.

Interactive Features: Add user controls to pause, step through, or reverse execution for better learning control.

Performance Metrics: Include time and space complexity analysis visualization to teach algorithm efficiency concepts.

Comparative Visualization: Display multiple data structures side-by-side to compare their behavior on the same task.

Export Functionality: Allow users to export reasoning steps and visualizations for documentation and study purposes.

Enhanced UI/UX: Improve the visual interface with animations, color coding, and interactive elements.

Integration with Learning Platforms: Develop APIs or plugins to integrate with educational platforms and learning management systems.

5.4 Broader Impact

This project contributes to computer science education by providing a tangible example of how theoretical DSA concepts apply to practical problem-solving. By making abstract concepts visible and interactive, it helps bridge the gap between classroom learning and real-world application, potentially improving student engagement and comprehension in DSA courses.

REFERENCES

[i] Ellis Horowitz, Sartaj Sahni, Susan Anderson-Freed, Fundamentals of Data Structures in C. Hyderabad, India: Universities Press, 2012.

[ii] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[iii] Mark Allen Weiss, Data Structures and Algorithm Analysis, Analysis, 3rd ed. Boston, MA, USA: Pearson Education, 2011.

[iv] Alfred V. Aho, John E. Hopcroft, Jeffrey D. Ullman, Data Structures and Algorithms, Addison-Wesley.

[v] Robert Lafore, Data Structures and Algorithms in Java, 2nd ed. Indianapolis, IN, USA: Pearson Education, 2003.

[vi] Adam Drozdek, Data Structures and Algorithms in C++, 4th ed. Boston, MA, USA: Cengage Learning, 2013.

[vii] Narasimha Karumanchi Data Structures and Algorithms Made Easy. Hyderabad, India: CareerMonk Publications, 2017.

[viii] Donald E. Knuth, The Art of Computer Programming, Vol. 1: Fundamental Algorithms, 3rd ed. Boston, MA, USA: Addison-Wesley, 1997.

[ix] Seymour Lipschutz, Data Structures, McGraw-Hill ,Data Structures. New York, NY, USA: McGraw-Hill, 2004.

[x] Stuart K. Card, Jock D. Mackinlay, Ben Shneiderman, Readings in Information Visualization: Using Vision to Think. San Francisco, CA, USA: Morgan Kaufmann, 1999.

[xi] Stuart Russell, Peter Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Upper Saddle River, NJ, USA: Pearson Education, 2010.

[xii] Brett Victor, "Learnable programming: Designing a programming system for understanding programs," *Communications of the ACM*, vol. 55, no. 6, pp. 30–33, Jun. 2012.

Appendix

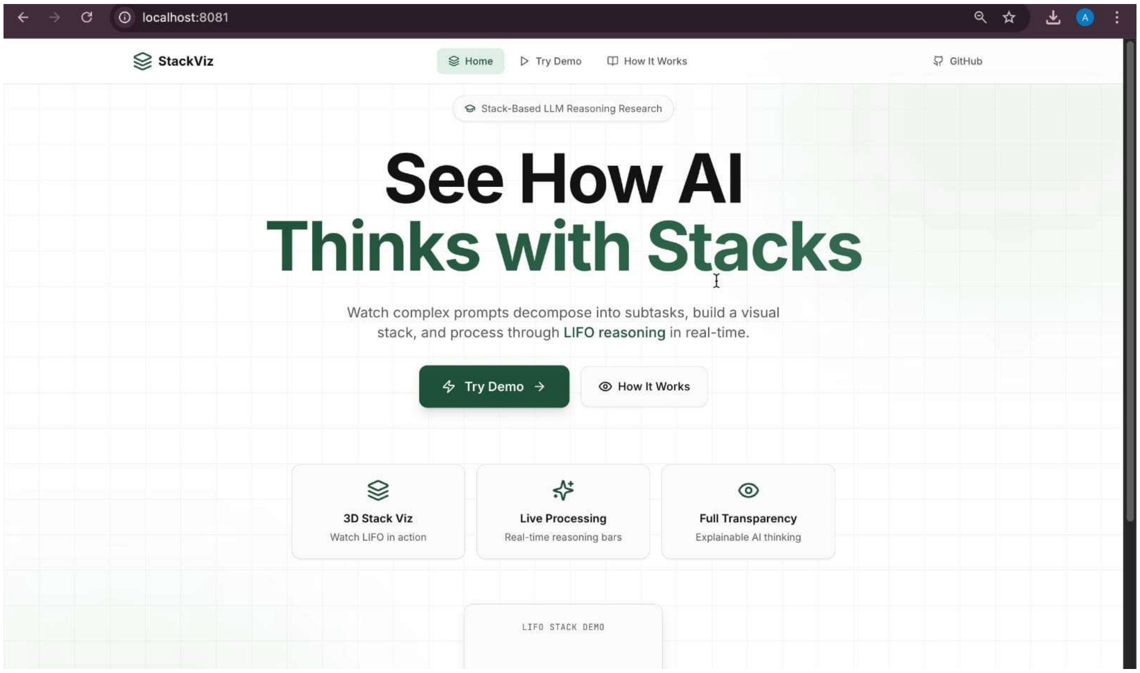

Fig.1 Website

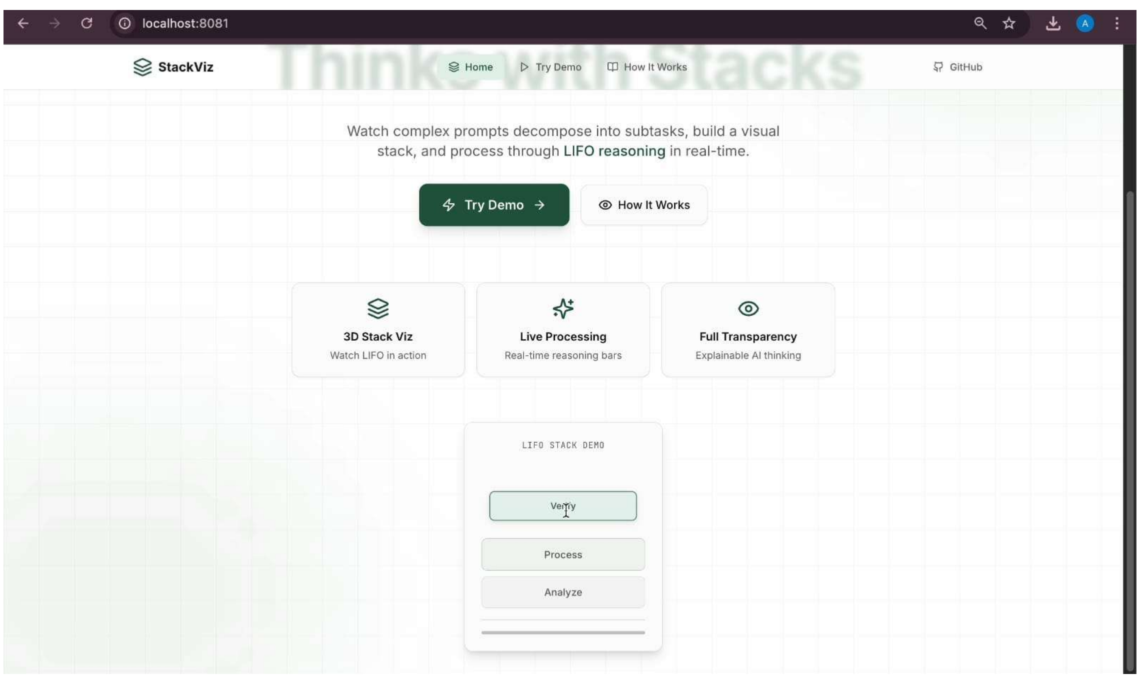

Fig. 2 LIFO Stack Demo

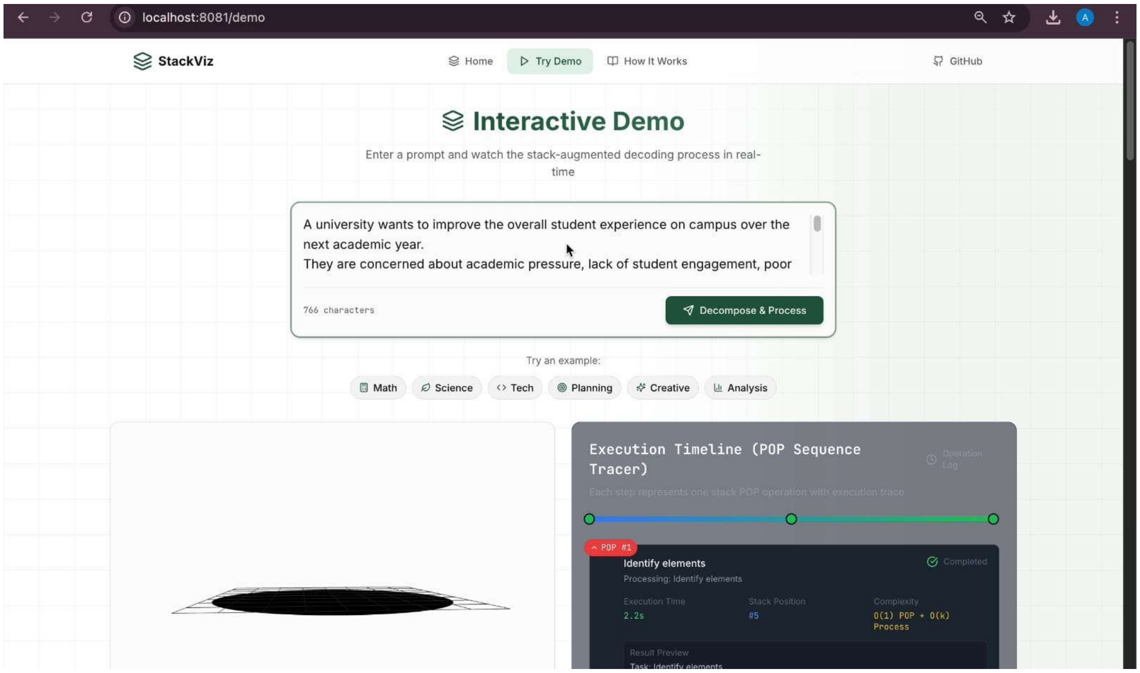

Fig. 3 Interactive Demo

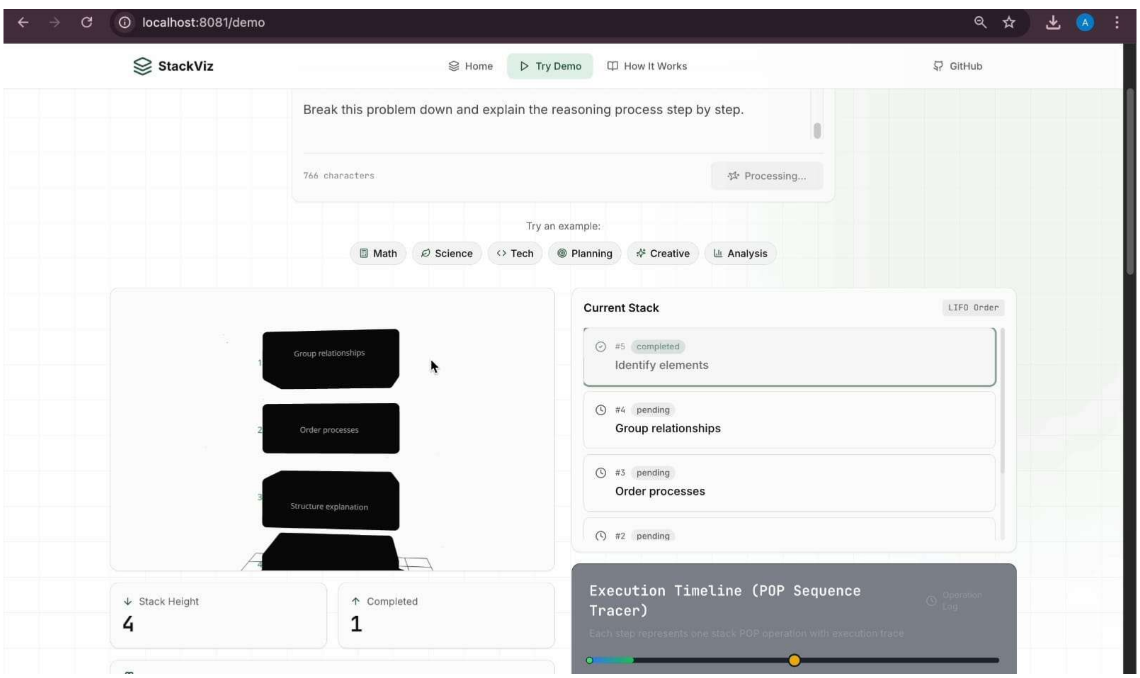

Fig. 4 Current Stack

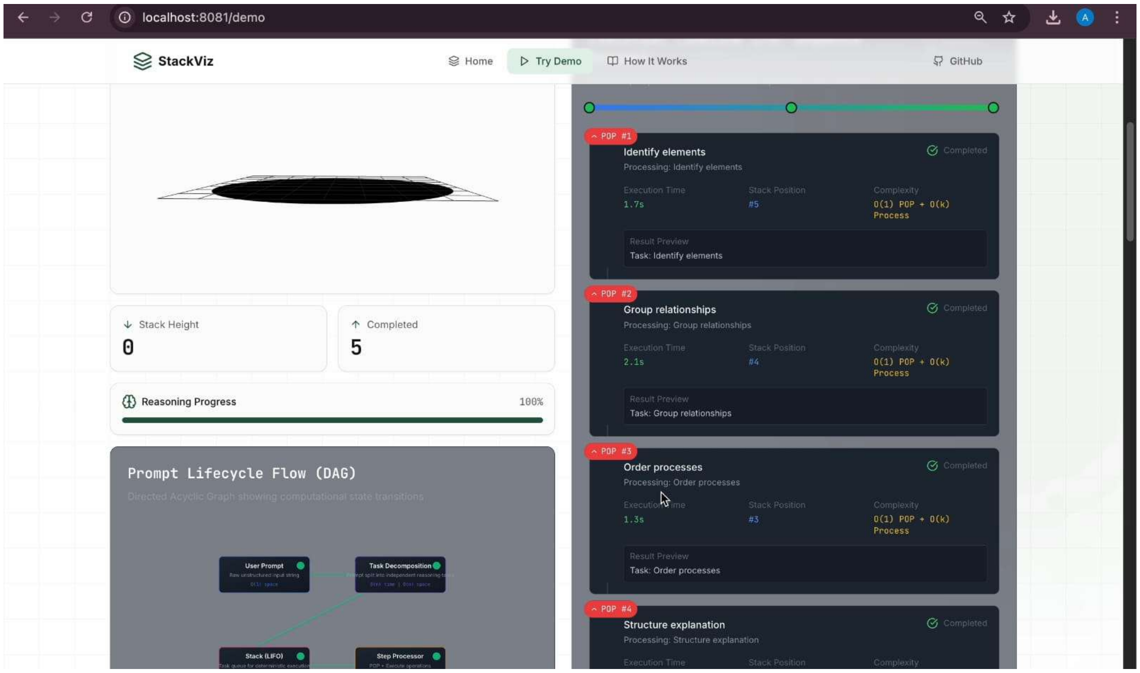

Fig. 5 Prompt Lifecycle Flow and Execution Timeline

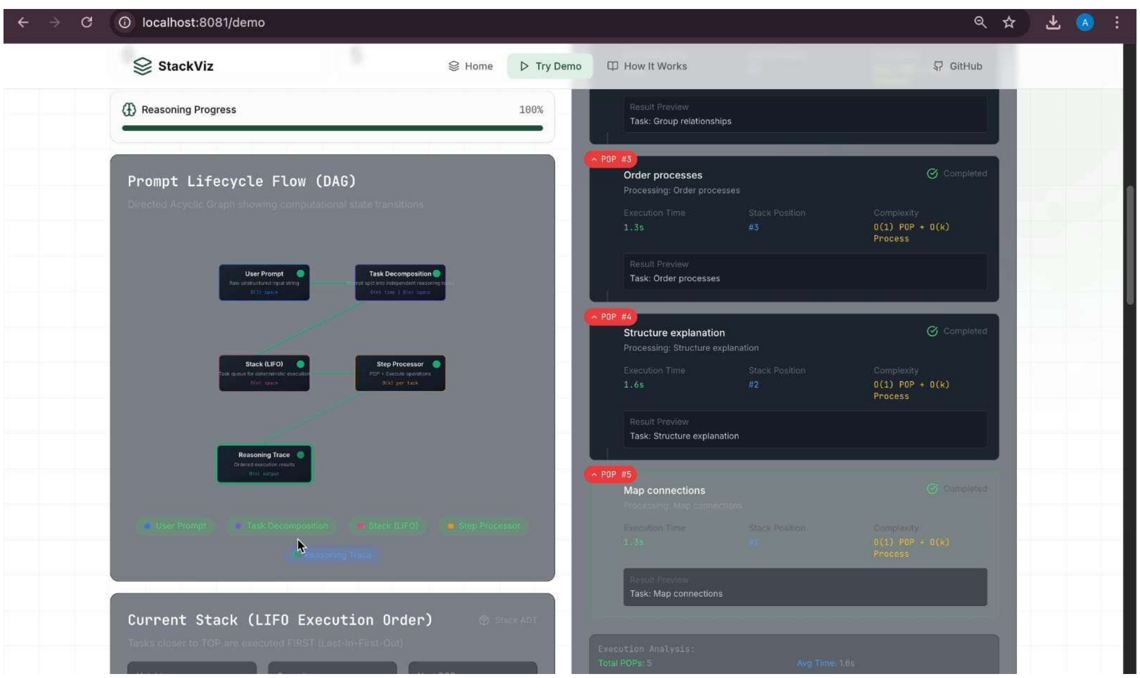

Fig. 6 Prompt Lifecycle Flow (DAG)

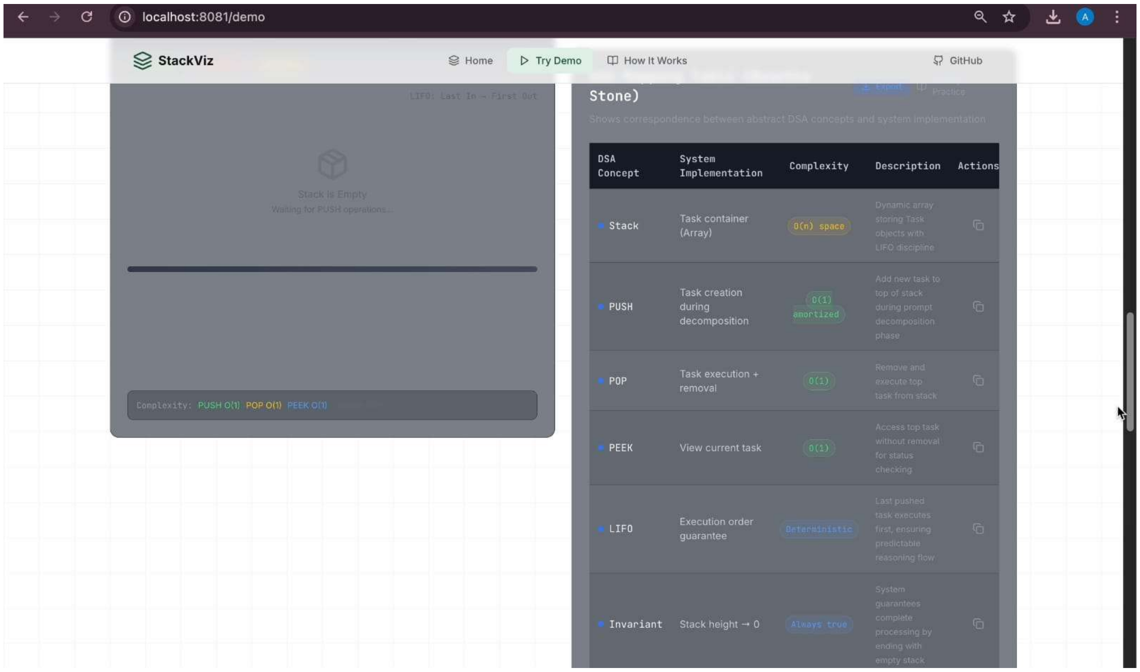

Fig. 7 DSA Mapping Table (Rosetta Stone)